

Chapter 3: Transport Layer

Our goals:

- ❑ understand principles behind transport layer services:
 - multiplexing/demultiplexing
 - reliable data transfer
 - flow control
 - congestion control
- ❑ learn about transport layer protocols in the Internet:
 - UDP: connectionless transport
 - TCP: connection-oriented transport with congestion control

10/14/2013

Transport Layer (SSL) 3-1

Chapter 3 outline

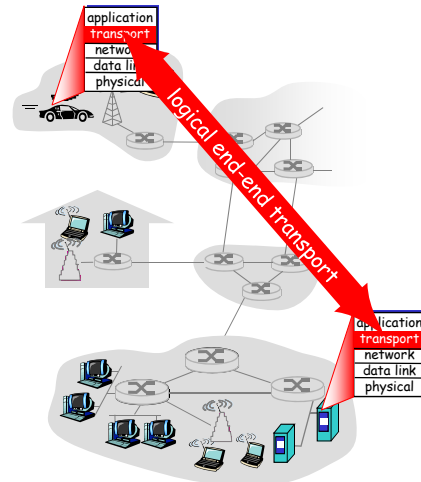
- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
(my slides for 3.4 do not follow Kurose & Ross)
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

10/14/2013

Transport Layer (SSL) 3-2

Transport services and protocols

- ❑ provide *logical communication* between **app processes** on different hosts
- ❑ transport protocols run in end systems (primarily)
 - send side: breaks app messages into **segments**, passes to network layer
 - rcv side: reassembles segments into messages, passes to app layer

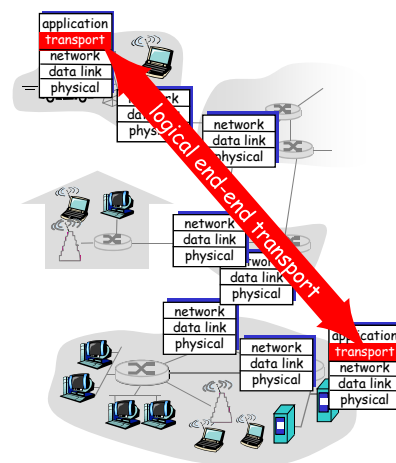


10/14/2013

Transport Layer (SSL) 3-3

Internet transport-layer protocols

- ❑ reliable, in-order byte delivery by **TCP**
 - congestion control
 - flow control
 - connection setup
- ❑ unreliable, unordered delivery by **UDP**
 - no-frills extension of "best-effort" IP
- ❑ services not available:
 - delay guarantees
 - bandwidth guarantees



10/14/2013

Transport Layer (SSL) 3-4

Chapter 3 outline

- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

10/14/2013

Transport Layer (SSL) 3-5

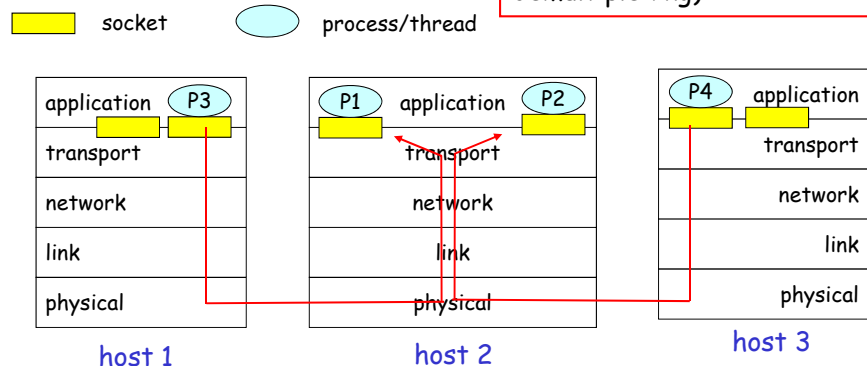
Multiplexing/demultiplexing

Demultiplexing at rcv host:

deliver received segments to correct sockets

Multiplexing at send host:

gather data from multiple sockets, encapsulate data with header (later used for demultiplexing)

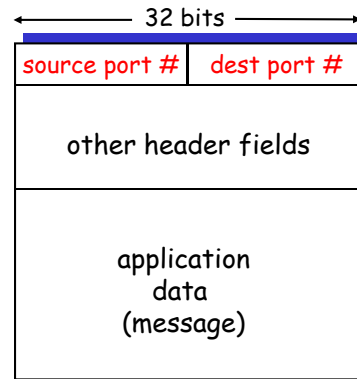


10/14/2013

Transport Layer (SSL) 3-6

How demultiplexing works

- host receives IP datagrams
- It uses IP addresses & port numbers to direct segment to appropriate socket



TCP/UDP segment format

10/14/2013

Transport Layer (SSL) 3-7

Connectionless demultiplexing

- UDP socket identified by two-tuple:
(dest IP address, dest port number)
- IP datagrams from *different sources* directed to *same* UDP socket
- When host receives UDP segment:
 - directs UDP segment to socket with destination port number

10/14/2013

Transport Layer (SSL) 3-8

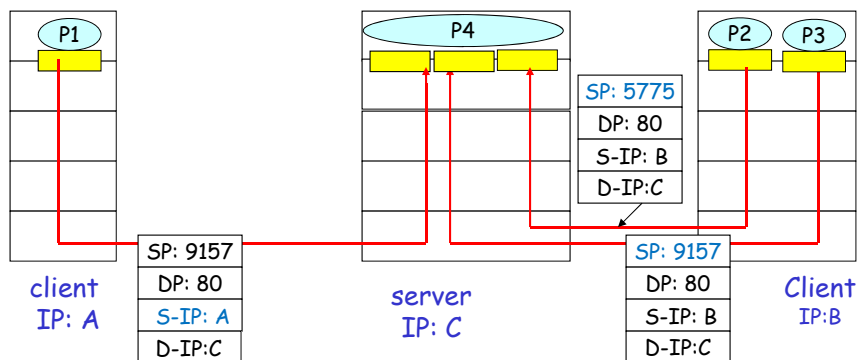
Connection-oriented demux

- ❑ Server has **welcome** and **connection** sockets
 - welcome socket is identified by server's IP address and a port number
- ❑ TCP connection socket identified by **4-tuple**:
 - source IP address
 - source port number
 - dest IP address
 - dest port number
- ❑ Server may support many simultaneous TCP connection sockets with clients:
 - each connection socket and the welcome socket have the *same port* number in server host
 - receiving host uses *all four values* to direct segment to appropriate connection socket

10/14/2013

Transport Layer (SSL) 3-9

Connection-oriented demux (cont)



10/14/2013

Transport Layer (SSL) 3-10

Chapter 3 outline

- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

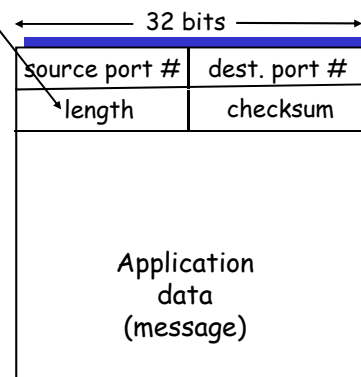
10/14/2013

Transport Layer (SSL) 3-11

UDP: User Datagram Protocol [RFC 768]

- ❑ "best effort" service, UDP segments (aka datagrams) may be:
 - lost
 - delivered out of order to appl
- ❑ *connectionless*:
 - no handshaking between UDP sender, receiver
 - each UDP segment handled independently of others

Length, in bytes of UDP segment including header



UDP segment format

10/14/2013

Transport Layer (SSL) 3-12

UDP (more)

- ❑ suitable for streaming multimedia applications
 - loss tolerant
 - rate sensitive
- ❑ other UDP uses, e.g.
 - DNS
 - SNMP
- ❑ reliable transfer over UDP?
 - add reliability in application layer
 - application-specific error recovery

Advantages of UDP

- ❑ no connection establishment (which can add delay)
 - simple: no connection state at sender, receiver
- ❑ no congestion control: UDP can blast away as fast as desired
- ❑ small segment header

10/14/2013

Transport Layer (SSL) 3-13

Internet checksum

Sender:

- ❑ treat segment as a sequence of 16-bit integers (with checksum field initialized to zero)
- ❑ add integers using 1's complement arithmetic and take 1's complement of the sum
- ❑ put result as checksum value into UDP checksum field
- ❑ detail: pseudoheader consisting of protocol no., IP addresses, UDP length field (again) included in checksum calculation

Receiver:

- ❑ compute 1's complement sum of received segment (checksum field included)
- ❑ check if computed sum equals sixteen 1's:
 - NO - error detected
 - YES - no error detected

But maybe errors nonetheless? More later

....

10/14/2013

Transport Layer (SSL) 3-14

Internet Checksum Example

□ Notes

- In ones complement arithmetic, a negative integer $-x$ is represented as the complement of x , i.e., each bit of x is inverted
- When adding numbers, a carryout from the most significant bit needs to be added to the result

□ Example: add two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

10/14/2013

Transport Layer (SSL) 3-15

Chapter 3 outline

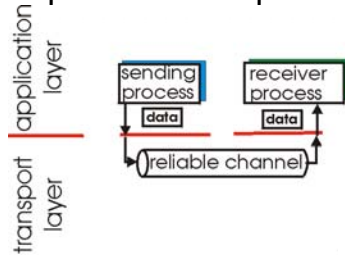
- 3.1 Transport-layer services
- 3.2 Multiplexing and demultiplexing
- 3.3 Connectionless transport: UDP
- 3.4 Principles of reliable data transfer
(my slides do not follow Kurose & Ross)
- 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- 3.6 Principles of congestion control
- 3.7 TCP congestion control

10/14/2013

Transport Layer (SSL) 3-16

Principles of Reliable data transfer

- important in application, transport, link layers
- top-10 list of important networking topics!



(a) provided service

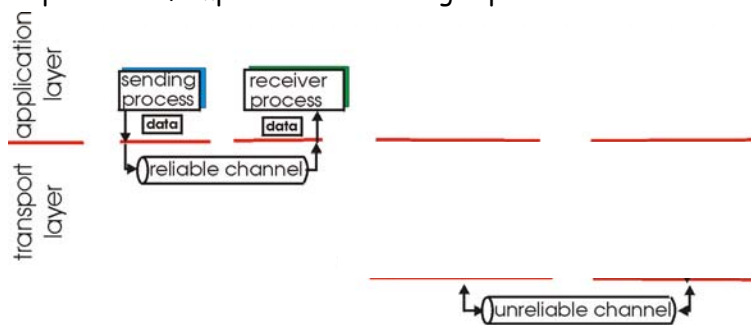
- characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

10/14/2013

Transport Layer (SSL) 3-17

Principles of Reliable data transfer

- important in app., transport, link layers
- top-10 list of important networking topics!



(a) provided service

(b) service implementation

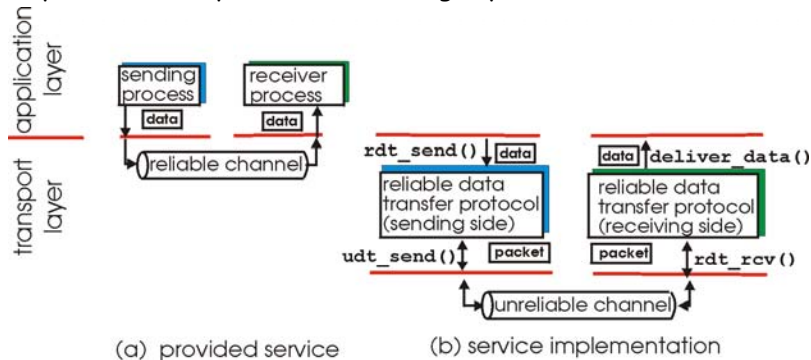
- characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

10/14/2013

Transport Layer (SSL) 3-18

Principles of Reliable data transfer

- important in app., transport, link layers
- top-10 list of important networking topics!



- characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

10/14/2013

Transport Layer (SSL) 3-19

Channel Abstractions

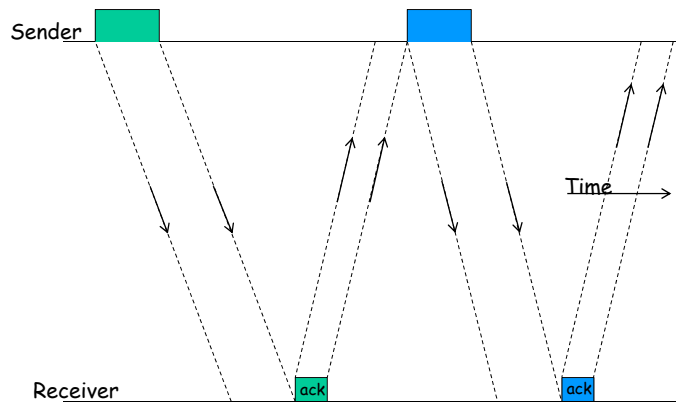
- **Lossy FIFO** channel
 - delivers a subsequence in FIFO order
 - example: delivery service provided by a **physical link**
- Lossy, reordering, duplicative (**LRD**) channel
 - example: delivery service provided by **IP** layer

10/14/2013

Transport Layer (SSL) 3-20

Stop-and-wait ARQ

□ Error-free operation



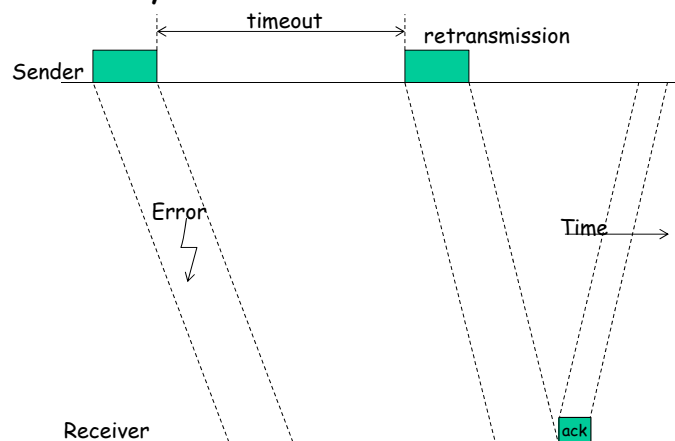
10/14/2013

Transport Layer (SSL) 3-21

Stop-and-wait ARQ

□ Retransmission after timeout

□ Recovery from loss of frame

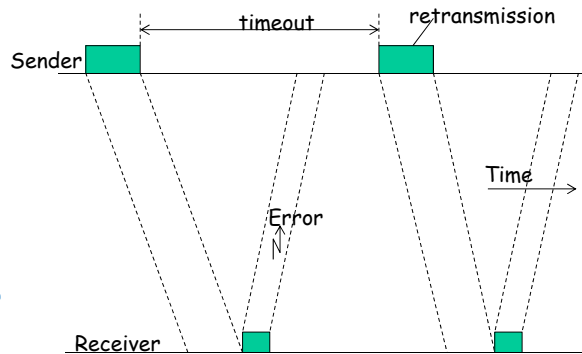


10/14/2013

Transport Layer (SSL) 3-22

Stop-and-wait ARQ

- ❑ Retransmission after timeout to recover from loss of ack
- ❑ Receiver gets duplicate frame
 - Sequence number needed in frame

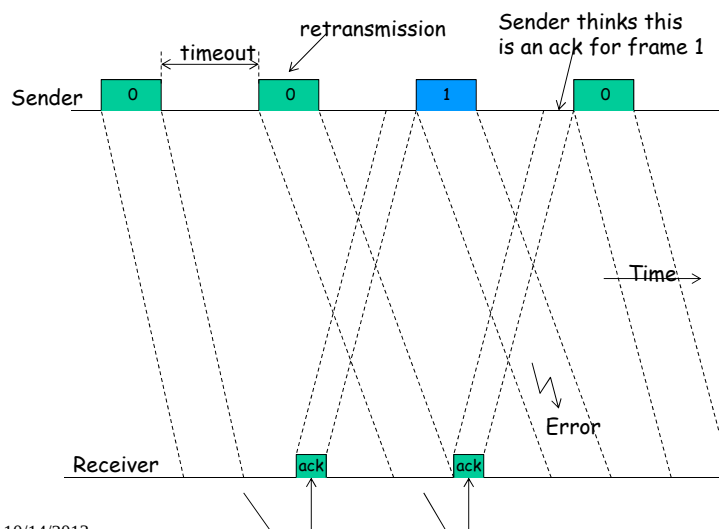


10/14/2013

Transport Layer (SSL) 3-23

Stop-and-wait ARQ

- ❑ Sequence number also needed in ack

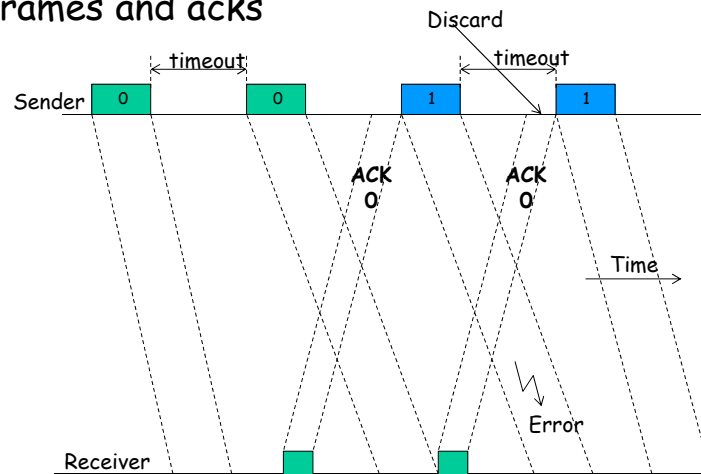


10/14/2013

Transport Layer (SSL) 3-24

Stop-and-wait ARQ

- Operation with 1-bit sequence numbers in frames and acks

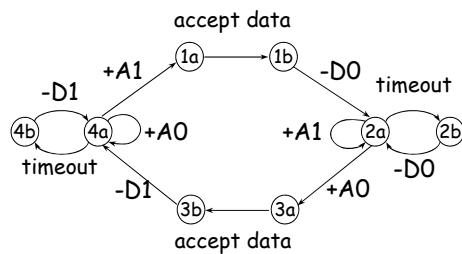


10/14/2013

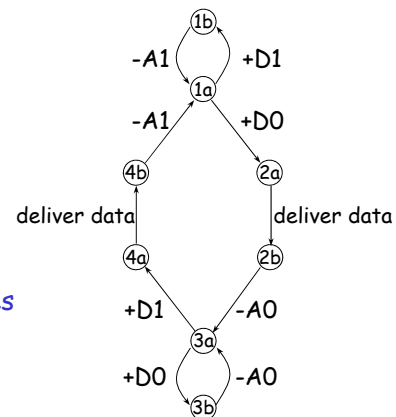
Transport Layer (SSL) 3-25

Alternating-Bit Protocol

Sender P1 (initial state = 1a)



Receiver P2 (initial state = 1a)



- Sender and Receiver specified by communicating finite-state machines

- Notation for arc labels

-m send message m

+m receive message m if it is waiting to be received

10/14/2013

Transport Layer (S. S. Lam) 3-26

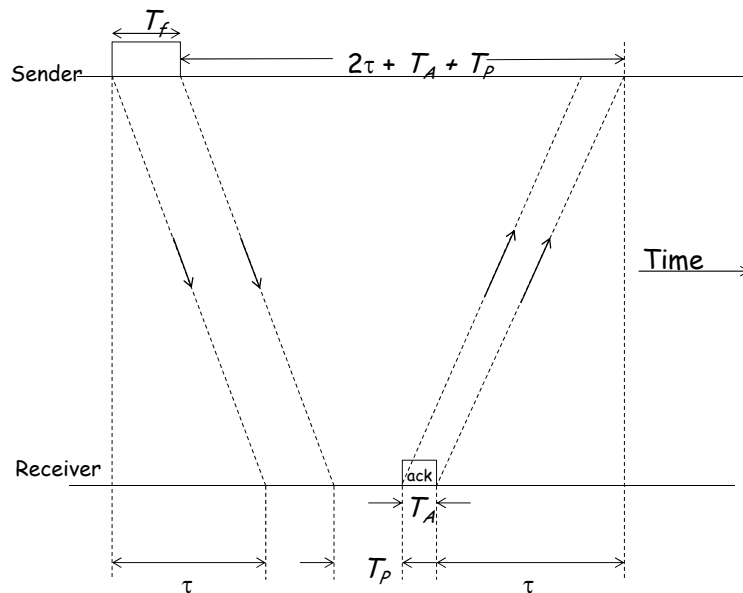
Alternating-Bit Protocol (cont.)

- ❑ **Assertion:** If Sender and Receiver communicate via lossy FIFO channels, the alternating-bit protocol provides **reliable in-order data delivery**.
- ❑ **Assumption:** A frame is retransmitted infinitely many times if it is lost infinitely many times.

10/14/2013

Transport Layer (SSL) 3-27

Stop-and-wait ARQ performance analysis



10/14/2013

Transport Layer (SSL) 3-28

P = probability transmission is unsuccessful

b_i = Prob[success after i transmissions] for $i = 1, 2, \dots$

$$b_i = P^{i-1}(1-P)$$

Average number of transmissions per frame

$$N_f = \sum_{i=1}^{\infty} i b_i = \sum_{i=1}^{\infty} i P^{i-1} (1-P)$$

$$= (1-P) \sum_{i=1}^{\infty} i P^{i-1}$$

$$= (1-P) \frac{d}{dP} \sum_{i=1}^{\infty} P^i = (1-P) \frac{d}{dP} \sum_{i=0}^{\infty} P^i$$

$$= (1-P) \frac{d}{dP} \frac{1}{1-P} = (1-P) \frac{1}{(1-P)^2}$$

$$= \frac{1}{1-P} = N_f$$

10/14/2013

Transport Layer (SSL) 3-29

Timeout duration $T > 2\tau + T_p + T_A$

Each unsuccessful transmission uses

$$T_f + T$$

Each successful transmission uses

$$T_f + 2\tau + T_p + T_A$$

Average time per frame

$$(N_f - 1) (T_f + T) + (T_f + 2\tau + T_p + T_A)$$

Max. utilization (throughput) of stop-and-wait

$$U = \frac{T_f}{\frac{P}{1-P} (T_f + T) + T_f + 2\tau + T_p + T_A}$$

10/14/2013

Transport Layer (SSL) 3-30

Effect of propagation delay & transmission time

Assume $P = 0$, $T_A = 0$, $T_P = 0$

$$\begin{aligned} U &\cong \frac{T_f}{T_f + 2\tau} && \text{(upper bound)} \\ &= \frac{1}{1 + \frac{2\tau}{T_f}} = \frac{1}{1 + 2a} && \text{where } a = \frac{\tau}{T_f} \end{aligned}$$

Note:

$$\tau = \frac{\text{distance}}{\text{propagation speed}}$$

$$T_f = \frac{\text{frame length}}{\text{transmission rate}}$$

10/14/2013

Transport Layer (SSL) 3-31

Performance of ABP

- ABP works, but performance degrades for channels with large *delay-bandwidth* product
- example: 1 Gbps link, 15 ms prop. delay, 1KByte packet

$$T_{\text{transmit}} = \frac{8\text{Kbits}}{10^9 \text{ bits/sec}} = 8 \text{ microsec}$$

$$U = \frac{8 \text{ microsec}}{30008 \text{ microsec}} = 0.00027$$

- protocol limits use of available bandwidth

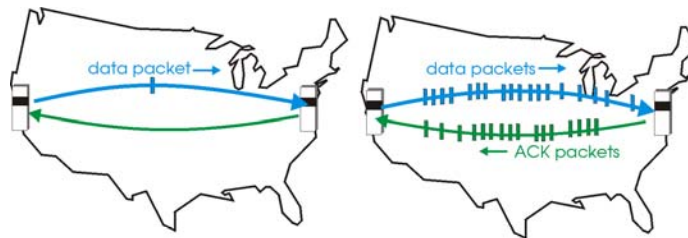
10/14/2013

Transport Layer (SSL) 3-32

Pipelined protocols

Pipelining: sender allows multiple, "in-flight", yet-to-be-acknowledged packets

- range of sequence numbers must be increased
- buffering at sender and/or receiver



(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

- Pipelined protocols: (i) concurrent logical channels, (ii) sliding window protocol

10/14/2013

Transport Layer (SSL) 3-33

Concurrent Logical Channels

- multiplex many logical channels on a link; run ABP for each logical channel (in both directions)
- each side maintains 3 bits of state for each channel indicating
 - ❖ channel busy
 - ❖ seq number sent
 - ❖ seq number expecting to receive
- ARPANET supported 8 logical channels on each ground link, 16 on each satellite link
 - header of each frame included a 3-bit channel number and a 1-bit sequence number
- This technique provides reliability only -- unordered delivery, and no flow control

10/14/2013

Transport Layer (S. S. Lam) 3-34

- Consider an infinite array, **Source**, at the sender, and an infinite array, **Sink**, at the receiver.



Sliding Windows in Action

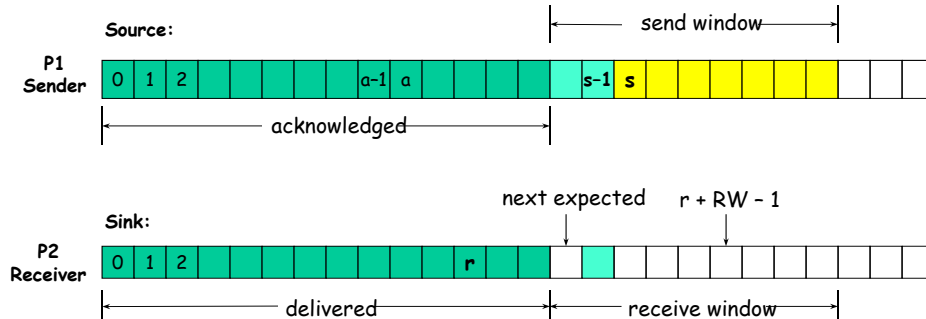
- ❑ Data unit **r** has just been received by P2
 - Receive window slides forward
- ❑ P2 sends **cumulative ack** with sequence number it expects to receive next (**r+3**)



Transport Layer (S. S. Lam) 3-36

Sliding Windows in Action

- P1 has just received cumulative ack with $r+3$ as next expected sequence number
 - Send window slides forward



10/14/2013

Transport Layer (S. S. Lam) 3-37

Sliding Window protocol

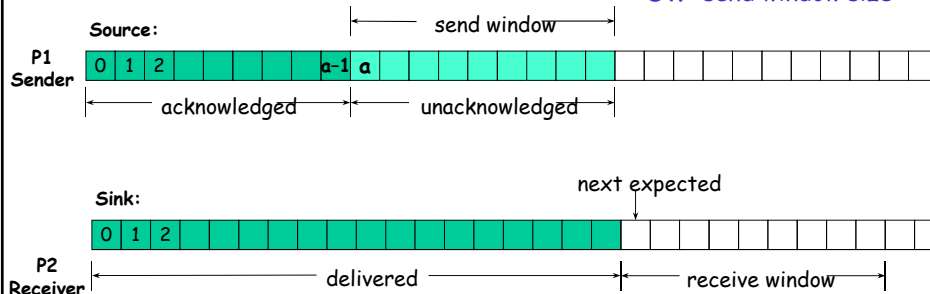
- Functions provided
 - error control (reliable delivery)
 - in-order delivery
 - flow and congestion control (by varying send window size)
- TCP uses cumulative acks (needed for correctness)
- Other kinds of acks (to improve performance)
 - selective nack
 - selective ack (TCP SACK)
 - bit-vector representing entire state of receive window (in addition to first sequence number of window)

10/14/2013

Transport Layer (S. S. Lam) 3-38

Sliding Windows for Lossy FIFO Channels

- A small number of bits in packet header for sequence number
- Necessary and sufficient condition for correct operation: $SW + RW \leq \text{MaxSeqNum}$
- Necessity:



10/14/2013

Transport Layer (S. S. Lam) 3-39

Sliding Windows for Lossy FIFO Channels

- Sufficiency can only be demonstrated by using a formal method to prove that the protocol provides reliable in-order delivery. See Shankar and Lam, *ACM TOPLAS*, Vol. 14, No. 3, July 1992.
- Interesting special cases
 - $SW = RW = 1$
alternating-bit protocol
 - $SW = 7, RW = 1$
out-of-order arrivals not accepted, e.g., HDLC
 - $SW = RW$

10/14/2013

Transport Layer (S. S. Lam) 3-40

Sliding Windows for LRD Channels

- ❑ **Assumption:** Packets have bounded lifetime L
- ❑ Be careful how fast sequence numbers are consumed (i.e., data are generated for delivery)
(consumption rate) $L < \text{MaxSeqNum}$
- ❑ TCP
 - 32-bit sequence numbers
 - counts bytes
 - assumes that datagrams will be discarded by IP if too old

10/14/2013

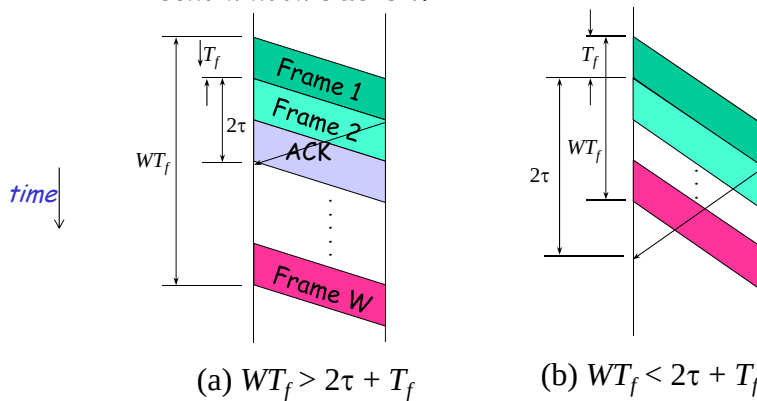
Transport Layer (S. S. Lam) 3-41

Sliding Window Protocol Performance Analysis

❑ Assumptions

[Go to slide 3-47](#)

- ack transmission time is negligible, $T_A = 0$
- receiver processing time is negligible, $T_p = 0$
- send window size is W



10/14/2013

Transport Layer (S. S. Lam) 3-42

Performance for Error-Free Channels

□ Maximum utilization

$$U = \begin{cases} 1 & WT_f > 2\tau + T_f \\ \frac{WT_f}{T_f + 2\tau} & WT_f \leq 2\tau + T_f \end{cases}$$

□ Define $a = \tau/T_f$

$$U = \begin{cases} 1 & W > 2a + 1 \\ \frac{W}{1 + 2a} & W \leq 2a + 1 \end{cases}$$

10/14/2013

Transport Layer (S. S. Lam) 3-43

Performance Analysis for Error-Prone Channels

□ Define

N_f = Average number of transmissions per frame

□ Maximum utilization

$$U = \begin{cases} \frac{1}{N_f} & W > 2a + 1 \\ \frac{W/N_f}{1 + 2a} & W \leq 2a + 1 \end{cases}$$

□ To determine N_f for two cases

- Selective repeat (optimistic performance)
- Go-back-N (pessimistic performance)

10/14/2013

Transport Layer (S. S. Lam) 3-44

Performance Analysis of Error-Prone Channels

P = probability a transmission is unsuccessful

- Selective repeat (-> upper bound on U)

$$N_f = \frac{1}{1-P}$$

$$U = \begin{cases} 1-P & W > 2a+1 \\ \frac{W(1-P)}{1+2a} & W \leq 2a+1 \end{cases}$$

- Go-back-N (-> lower bound on U)

Each lost frame requires the retransmission of N frames where $0 < N \leq W$

10/14/2013

Transport Layer (S. S. Lam) 3-45

Go-back-N (cont.)

- With probability $P^i(1-P)$, a frame requires $iN+1$ transmissions to succeed, for $i=0,1,\dots$

$$N_f = \sum_{i=0}^{\infty} (1+iN) P^i (1-P)$$

$$= \sum_{i=0}^{\infty} P^i (1-P) + NP(1-P) \sum_{i=0}^{\infty} i P^{i-1}$$

$$= 1 + NP(1-P) \frac{d}{dP} \sum_{i=0}^{\infty} P^i$$

$$= 1 + NP(1-P) \frac{d}{dP} \frac{1}{1-P}$$

$$= 1 + NP(1-P) \frac{1}{(1-P)^2}$$

$$= 1 + \frac{NP}{1-P}$$

10/14/2013

Transport Layer (S. S. Lam) 3-46

Got to slide 3-42

Go-back-N (cont.)

From previous slide

For $WT_f > 2\tau + T_f$

$$NT_f \cong T_f + 2\tau$$

$$N \cong 1 + 2a$$

$$N_f = 1 + \frac{NP}{1-P}$$

$$N_f = 1 + \frac{(1+2a)P}{1-P} = \frac{1-P+P+2aP}{1-P} = \frac{1+2aP}{1-P}$$

For $WT_f \leq 2\tau + T_f$

$$N = W$$

$$N_f = 1 + \frac{WP}{1-P} = \frac{1-P+WP}{1-P}$$

10/14/2013

Transport Layer (S. S. Lam) 3-47

Go-back-N (cont.)

From previous slide

□ Recall

$$U = \begin{cases} \frac{1}{N_f} & W > 2a+1 \\ \frac{W/N_f}{1+2a} & W \leq 2a+1 \end{cases}$$

$N_f = \frac{1+2aP}{1-P}$

$N_f = \frac{1-P+WP}{1-P}$

□ Maximum utilization

$$U = \begin{cases} \frac{1-P}{1+2aP} & W > 2a+1 \\ \frac{W(1-P)}{(1+2a)(1-P+WP)} & W \leq 2a+1 \end{cases}$$

10/14/2013

Transport Layer (S. S. Lam) 3-48

Chapter 3 outline

- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 **Connection-oriented transport: TCP**
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

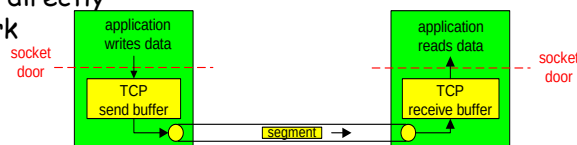
10/14/2013

Transport Layer (SSL) 3-49

TCP: Overview

RFCs: 793, 1122, 1323, 2018, 2581

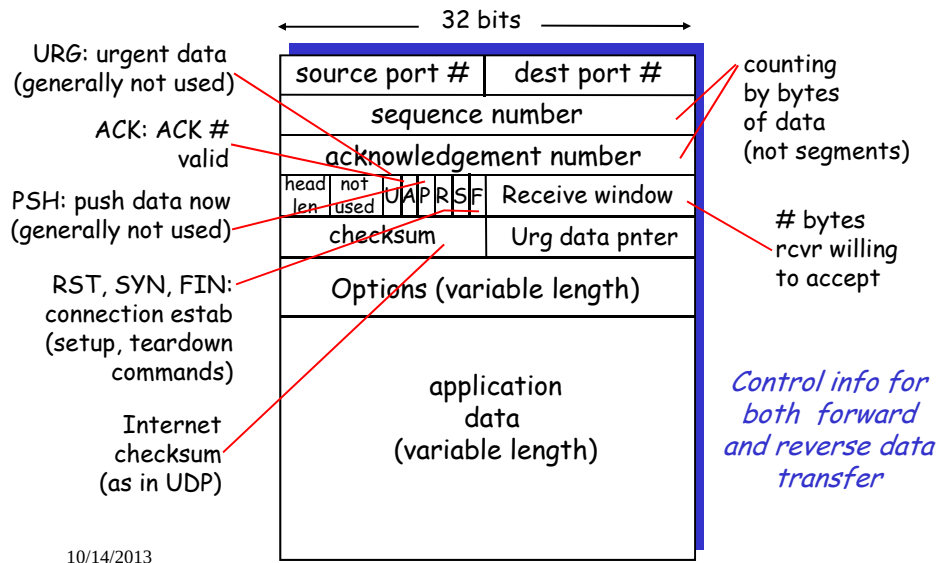
- ❑ **connection-oriented**
 - handshaking initializes sender, receiver state before data exchange
- ❑ **point-to-point**
 - two sender-receiver pairs
 - bi-directional data flows in same connection
- ❑ **MSS: maximum segment size**
 - ❖ less than MTU of directly connected network
- ❑ **reliable, in-order *byte stream service***
 - no "message boundaries"
 - send and receive buffers
- ❑ **pipelined**
 - send window size determined by TCP congestion and flow control



10/14/2013

Transport Layer (SSL) 3-50

TCP segment structure



TCP seq. #'s and ACKs

Seq.

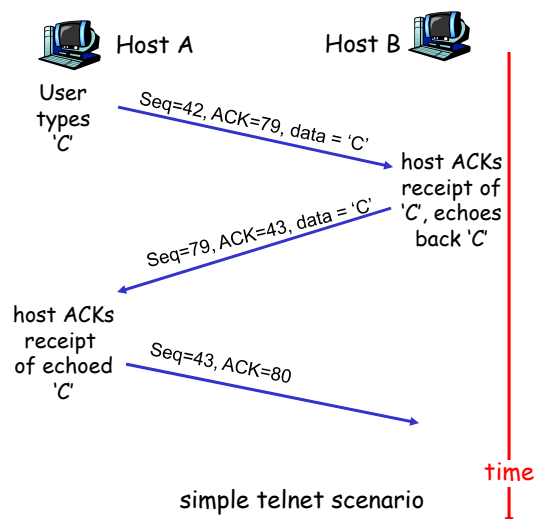
- sequence number of first byte in segment's data

ACK

- seq # of next byte expected from other side

cumulative ACK

- Q:** how receiver handles out-of-order segments?
TCP spec doesn't say, up to implementor



10/14/2013

Transport Layer (SSL) 3-52

TCP Round Trip Time and Timeout

Q: how to set TCP timeout value?

- ❑ longer than RTT
 - but RTT varies
- ❑ too short: premature timeout
 - unnecessary retransmissions
- ❑ too long: slow reaction to segment loss

Q: how to estimate RTT?

- ❑ **SampleRTT**: measured time from segment transmission until ACK receipt
 - ignore retransmissions
- ❑ **SampleRTT** will vary, want estimated RTT "smoother"
 - average several recent measurements, not just current **SampleRTT**

10/14/2013

Transport Layer (SSL) 3-53

TCP Round Trip Time and Timeout

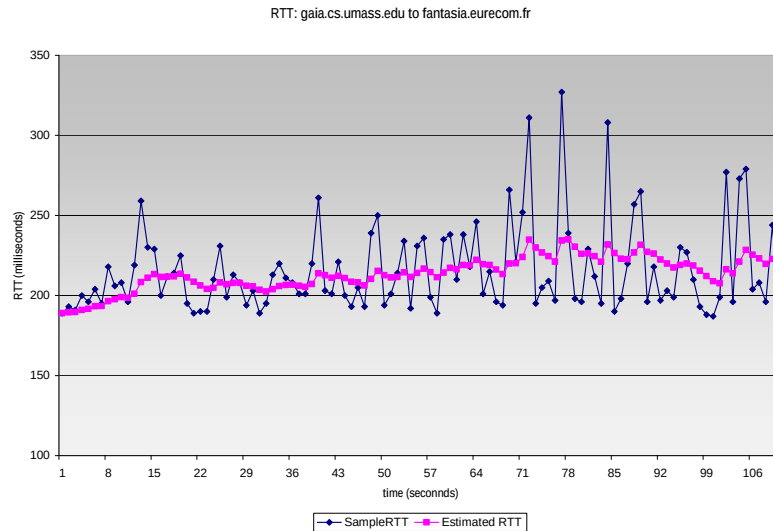
$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- ❑ Exponential weighted moving average
- ❑ influence of past sample decreases exponentially fast
- ❑ typical value: $\alpha = 0.125$

10/14/2013

Transport Layer (SSL) 3-54

Example RTT estimation:



Transport Layer (SSL) 3-55

Setting the timeout interval

- ❑ **EstimatedRTT** plus "safety margin"
 - large variation in **EstimatedRTT** -> larger safety margin
- ❑ first estimate of how much **SampleRTT** deviates from **EstimatedRTT**:
$$\text{DevRTT} = (1-\beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

(typically, $\beta = 0.25$)

Then set timeout interval:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$

10/14/2013

Transport Layer (SSL) 3-56

Chapter 3 outline

- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - **reliable data transfer**
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

10/14/2013

Transport Layer (SSL) 3-57

TCP reliable data transfer

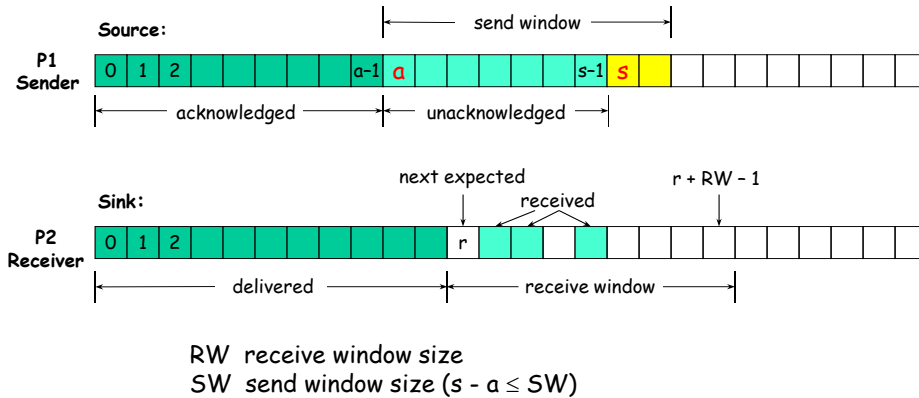
- ❑ TCP creates rdt service on top of IP's unreliable service
- ❑ Cumulative acks
- ❑ TCP uses single retransmission timer
- ❑ Retransmissions are triggered by:
 - timeout events
 - **duplicate acks**
- ❑ Initially consider simplified TCP sender:
 - ignore duplicate acks
 - ignore flow control, congestion control

10/14/2013

Transport Layer (SSL) 3-58

Sliding Window Protocol

At the sender, **a** will be pointed to by **SendBase**,
and **s** by **NextSeqNum**



10/14/2013

Transport Layer (S. S. Lam) 3-59

NextSeqNum = InitialSeqNum

SendBase = InitialSeqNum

loop (forever) {
 switch(event)

event: data received from application above
 and send window has enough room
 create TCP segment with sequence number NextSeqNum
 if (timer currently not running)
 start timer
 pass segment to IP
 NextSeqNum = NextSeqNum + length(data)

event: timer timeout
 retransmit not-yet-acknowledged segment with
 smallest sequence number
 start timer

event: ACK received, with ACK field value = y
 if (y > SendBase) {
 SendBase = y
 if (there are currently not-yet-acknowledged segments)
 start timer;
 else stop timer
 }
 } /* end of loop forever */

TCP
sender
(simplified)

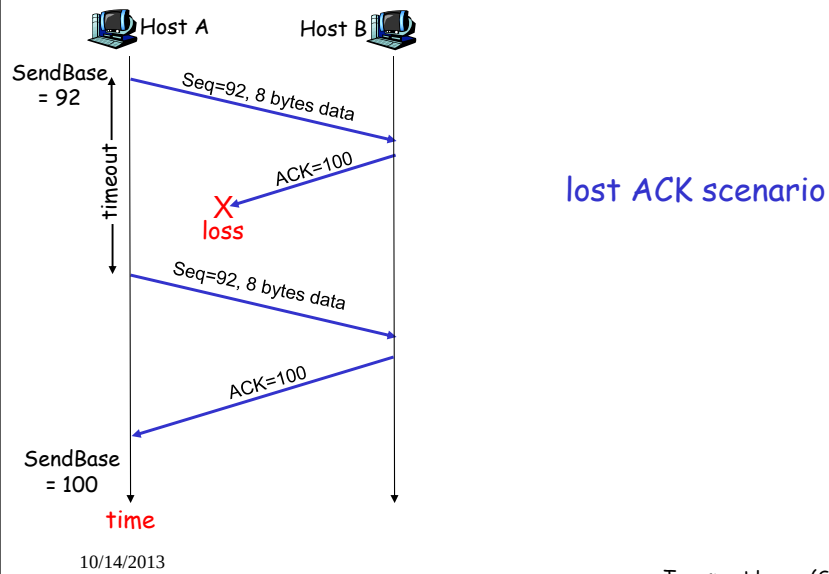
Note:

• $y > \text{SendBase}$
 means new data
 ack'ed

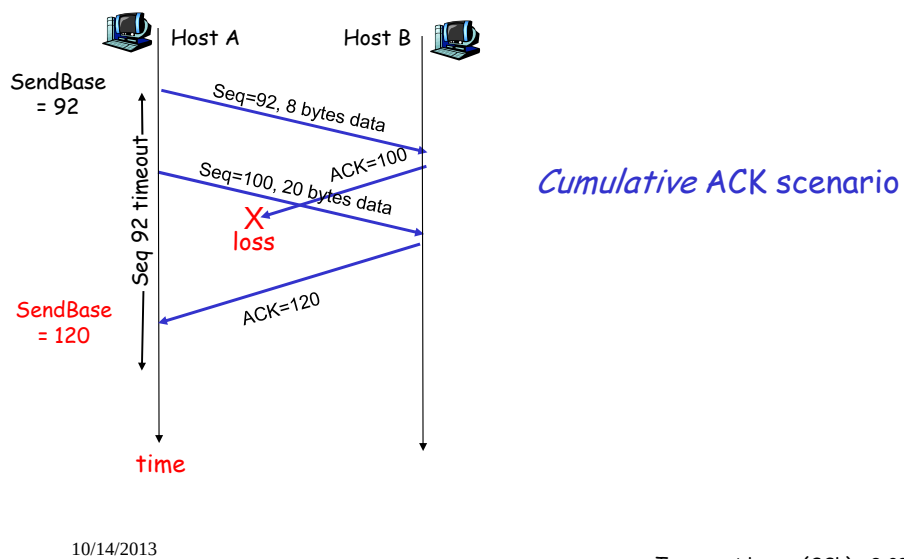
10/14/2013

Transport Layer (SSL) 3-60

TCP: retransmission scenario (1)

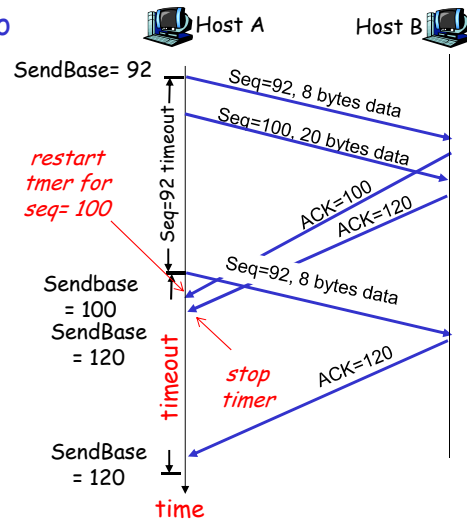


TCP retransmission scenario (2)



TCP: retransmission scenario (3)

premature timeout scenario



10/14/2013

Transport Layer (SSL) 3-63

TCP ACK generation [RFC 1122, RFC 2581]

Event at Receiver	TCP Receiver action
Arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	Delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK
Arrival of in-order segment with expected seq #. One other segment has ACK pending	Immediately send single cumulative ACK, ACKing both in-order segments
Arrival of out-of-order segment higher-than-expected seq. # . Gap detected	Immediately send <i>duplicate</i> ACK, indicating seq. # of next expected byte
Arrival of segment that partially or completely fills gap	Immediately send ACK, provided that segment starts at lower end of gap

10/14/2013

Transport Layer (SSL) 3-64

Fast Retransmit

- Time-out period often relatively long:
 - long delay before resending lost packet
- Detect lost segments via duplicate ACKs
 - Sender often sends many segments back-to-back
 - If segment is lost, there will likely be many duplicate ACKs.
- If sender receives 3 **duplicate ACKs** for the same data, it supposes that segment after ACKed data was lost:
 - **fast retransmit:** resend segment before timer expires

10/14/2013

Transport Layer (SSL) 3-65

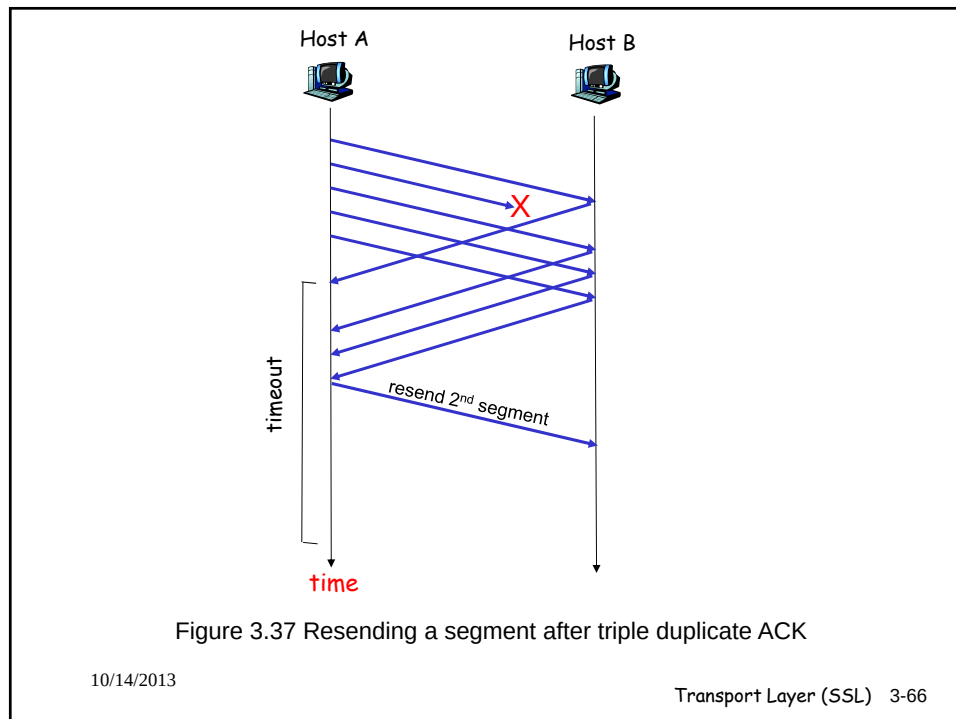


Figure 3.37 Resending a segment after triple duplicate ACK

10/14/2013

Transport Layer (SSL) 3-66

Fast retransmit algorithm:

```
event: ACK received, with ACK field value = y
    if (y > SendBase) {
        SendBase = y
        if (there is a not-yet-acknowledged segment)
            start timer
    }
    else {
        increment count of dup ACKs received for y
        if (count of dup ACKs received for y = 3) {
            resend segment with sequence number y
            reset timer for y
        }
    }
```

a duplicate ACK for already ACKed segment

fast retransmit

10/14/2013

Transport Layer (SSL) 3-67

Chapter 3 outline

- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

10/14/2013

Transport Layer (SSL) 3-68

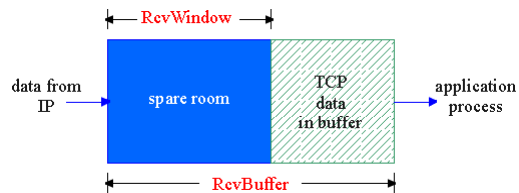
TCP Flow Control

flow control

sender won't overrun receiver's buffers by transmitting too much, too fast

receiver: explicitly informs sender of (dynamically changing) amount of free buffer space

- **RcvWindow** field in TCP segment



sender: keeps amount of transmitted, unACKed data less than most recently received **RcvWindow** value

buffer at receive side of a TCP connection

10/14/2013

Transport Layer (SSL) 3-69

Chapter 3 outline

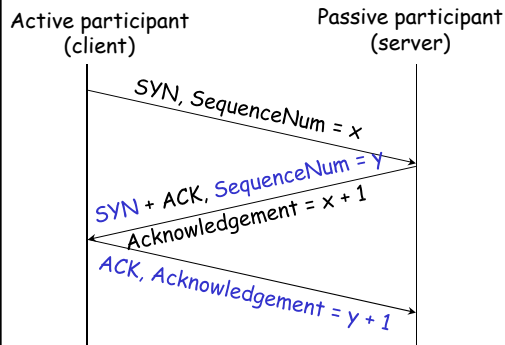
- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - **connection management**
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

10/14/2013

Transport Layer (SSL) 3-70

TCP Connection Management

- initialize TCP variables
 - seq. #s
 - buffers, flow control info (e.g. RcvWindow)



10/14/2013

Transport Layer (SSL) 3-71

Three way handshake

Step 1: client end system sends TCP SYN control segment to server - **initial seq number chosen at random**

Step 2: server end system receives SYN, replies with SYNACK control segment

- allocates buffers
- specifies server-to-receiver initial seq. # (chosen at random)

Step 3: client end system replies with ack # (may be piggybacked in data segment)

TCP Connection Management (cont.)

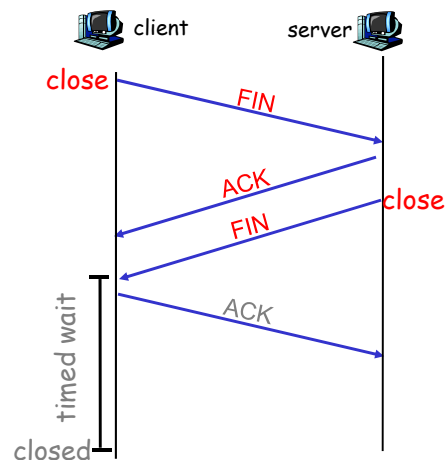
Closing a connection:

client closes socket

Step 1: client end system sends TCP **FIN** control message to server

Step 2: server receives FIN, replies with **ACK**.

Later no more data to send. It closes connection, sends **FIN**.



10/14/2013

Transport Layer (SSL) 3-72

TCP Connection Management (cont.)

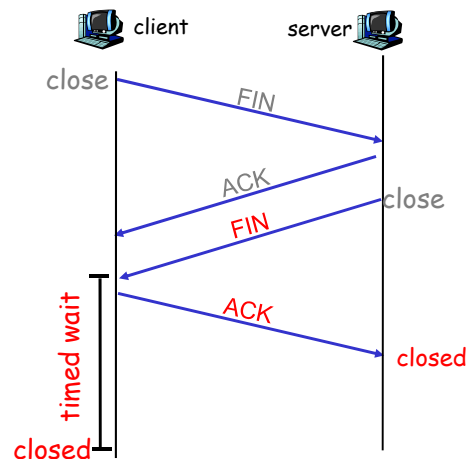
Step 3: client receives **FIN**,
replies with **ACK** and
enters "**timed wait**"

- will respond with **ACK**
to a retransmitted **FIN**
(due to lost of previous **ACK**)

Step 4: server receives
ACK. Its connection is
closed.

Step 5: client closes
connection at the end of
timed wait

Note: protocol spec allows
simultaneous **FINs**



10/14/2013

Transport Layer (SSL) 3-73

SYN Flood Attack

- ❑ A classic DoS attack - the sender sends a large number of TCP SYN segments, without completing the third step of the 3-way handshake
 - Server's connection resources are allocated for half-open connections.
 - Legitimate users are denied service
 - **SYN Cookie defense**
- ❑ DDoS - attack amplified by sending SYNs from multiple sources (botnet)

10/14/2013

Transport Layer (SSL) 3-74

Chapter 3 outline

- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

10/14/2013

Transport Layer (SSL) 3-75

Principles of Congestion Control

Congestion:

- ❑ informally: "too many sources sending too much data too fast for *network* to handle"
- ❑ different from flow control
- ❑ manifestations:
 - long delays (queueing in router buffers)
 - lost packets (buffer overflow at routers)
- ❑ a top-10 problem!

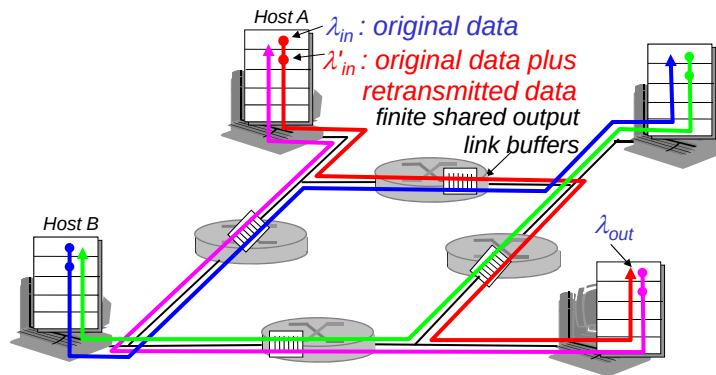
10/14/2013

Transport Layer (SSL) 3-76

Causes/costs of congestion: scenario

- ❑ four senders
- ❑ multi-hop paths
- ❑ Timeout & retransmit

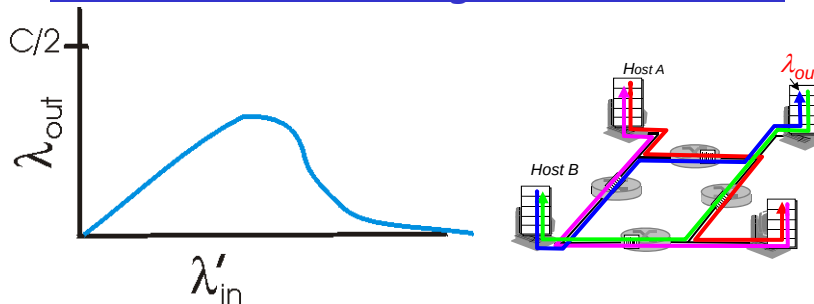
Q: what happens as λ_{in} and λ'_{in} increase at every sender?



10/14/2013

Transport Layer (SSL) 3-77

Causes/costs of congestion: scenario



Cost of congestion

- ❖ when a packet is dropped, any upstream transmission capacity used for that packet was wasted
- ❖ behavior on right side of above graph called **congestion collapse**

10/14/2013

Transport Layer (SSL) 3-78

Approaches towards congestion control

End-to-end congestion control:

- ❑ no explicit feedback from network
- ❑ congestion inferred from end-system's observed loss (or delay)
- ❑ approach taken by TCP

Network-assisted congestion control:

- ❑ routers provide feedback to end systems
 - single bit indicating congestion (SNA, DECbit, TCP/IP ECN, ATM)
 - explicit sending rate for sender

10/14/2013

Transport Layer (SSL) 3-79

Chapter 3 outline

- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

10/14/2013

Transport Layer (SSL) 3-80

TCP Congestion Control

- end-to-end control (no network assistance)
- sender limits transmission:
 $\text{LastByteSent} - \text{LastByteAcked} \leq \text{CongWin}$
- Roughly,

$$\text{send rate} \leq \frac{\text{CongWin}}{\text{RTT}} \text{ bytes/sec}$$

where CongWin is in bytes

How does sender determine CongWin?

- loss event = **timeout** or **3 duplicate acks**
- TCP sender reduces **CongWin** after loss event

three mechanisms:

- slow start
- reduce to 1 segment after timeout event
- AIMD

Note: Consider RcvWindow to be very large such that the send window size is equal to CongWin. They are referred to as rwnd and cwnd, respectively, in the textbook.

10/14/2013

Transport Layer (SSL) 3-81

TCP Slow Start

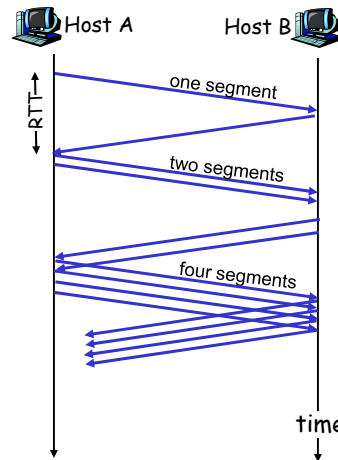
- Probing for usable bandwidth
- When connection begins, **CongWin = 1 MSS**
 - Example: MSS = 500 bytes & RTT = 200 msec
 - initial rate = 2500 bytes/sec = 20 kbps
- available bandwidth may be $\gg \text{MSS}/\text{RTT}$
 - desirable to quickly ramp up to a higher rate

10/14/2013

Transport Layer (SSL) 3-82

TCP Slow Start (more)

- When connection begins, increase rate exponentially **until first loss event or "threshold"**
 - double CongWin every RTT
 - done by incrementing CongWin by 1 MSS for every ACK received
- Summary: initial rate is slow but ramps up exponentially fast



10/14/2013

Transport Layer (SSL) 3-83

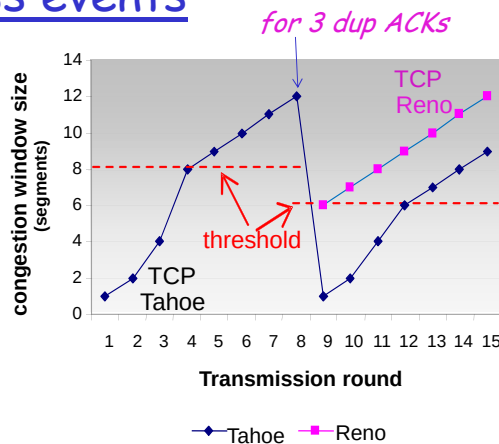
Congestion avoidance state & responses to loss events

Q: If no loss, when should the exponential increase switch to linear?

A: When CongWin gets to current value of **threshold**

Implementation:

- For initial slow start, threshold is set to a very large value (e.g., 64 Kbytes)
- Subsequently, threshold is variable
- At loss event, threshold is set to 1/2 of CongWin just before loss event



Notes: 1. For simplicity, CongWin is in number of segments in the above graph. 2. Reno's window inflation/deflation steps omitted

10/14/2013

Transport Layer (SSL) 3-84

Rationale for Reno's Fast Recovery

□ 3 dup ACKs indicate network capable of delivering some segments

□ timeout occurring before 3 dup ACKs is "more alarming"

- After 3 dup ACKs:
 - CongWin is cut in half (*multiplicative decrease*)
 - window then grows linearly (*additive increase*)
- But after timeout event:
 - CongWin is set to 1 MSS instead;
 - window then grows exponentially to a threshold, then grows linearly

Additive Increase Multiplicative Decrease (AIMD)

10/14/2013

Transport Layer (SSL) 3-85

Summary: TCP Congestion Control (Reno)

- When CongWin is below Threshold, sender in **slow-start** phase, window grows exponentially (until loss event or exceeding threshold).
- When CongWin is above Threshold, sender is in **congestion-avoidance** phase, window grows linearly.
- When a **triple duplicate ACK** occurs, Threshold set to CongWin/2 and CongWin set to Threshold.
- When **timeout** occurs, Threshold set to CongWin/2 and CongWin is set to 1 MSS.

10/14/2013

Transport Layer (SSL) 3-86

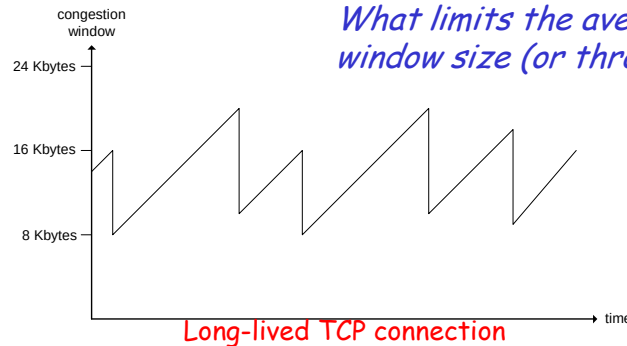
AIMD in steady state (no timeout)

additive increase:

increase **CongWin** by
1 MSS every RTT in
the absence of any
loss event: *probing*

multiplicative decrease:

cut **CongWin** in half
after loss event (3 dup
acks)



10/14/2013

Transport Layer (SSL) 3-87

TCP Throughput limited by loss rate

- TCP average throughput under AIMD (approximate) in terms of loss rate, L :

$$\text{throughput} = \frac{1.22 \times \text{MSS}}{\text{RTT} \sqrt{L}} \text{ bytes/second}$$

where MSS is number of bytes per segment

- Example: 1500-byte segments, 100ms RTT, to get 10 Gbps throughput, loss rate needs to be very low

$$L = 2 \cdot 10^{-10}$$

- New version of TCP needed for high-speed application

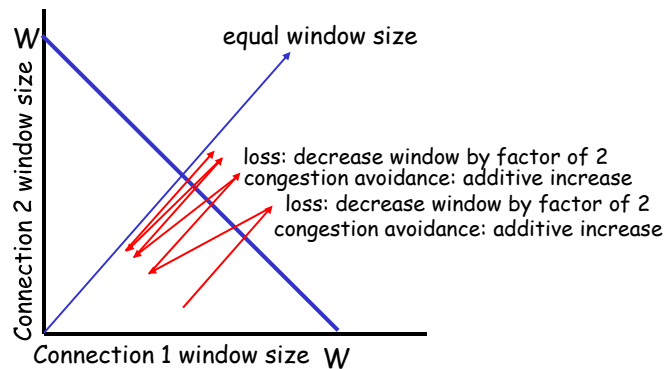
10/14/2013

Transport Layer (SSL) 3-88

Is TCP fair?

Two competing sessions:

- Additive increase gives slope of 1, as window size increases
- multiplicative decrease reduces window size to half (proportionally)

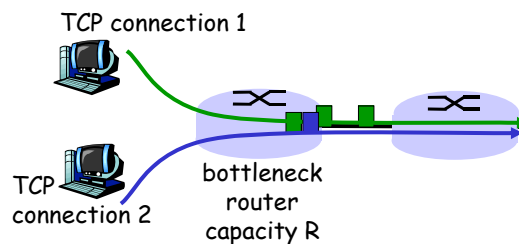


10/14/2013

Transport Layer (SSL) 3-89

Is TCP fair?

Fairness goal: if K TCP sessions share same bottleneck link of bandwidth R , each should have average rate of R/K



AIMD only provides convergence to same window size, not necessarily same throughput rate

10/14/2013

Transport Layer (SSL) 3-90

No fairness in practice

UDP

- Some multimedia apps use UDP instead of TCP
 - Can tolerate packet loss,
 - do not want rate throttled by congestion control - pump audio/video at constant rate

Parallel TCP connections

- nothing prevents an app from opening parallel connections between 2 hosts.
- Web browsers do this
 - Example: link of rate R supporting 9 connections
 - new app asks for 1 TCP, gets rate $R/10$
 - new app asks for 9 TCPs, gets rate $R/2$

10/14/2013

Transport Layer (SSL) 3-91

Chapter 3: Summary

- principles behind transport layer services:
 - multiplexing, demultiplexing
 - reliable data transfer
 - flow control
 - congestion control
- instantiation and implementation in the Internet
 - UDP
 - TCP

Next:

- leaving the network "edge" (application, transport layers)
- into the network "core"

10/14/2013

Transport Layer (SSL) 3-92